Consider the complex query: "What is the current temperature in all rooms on the second floor?"

With a RESTful API the client must make multiple sequential requests:

1. Send `GET /floors/2/rooms` to retrieve all rooms on the second floor.
2. For each room, send `GET /rooms/{id}/sensors` to find sensors in that room.
3. For each sensor, `GET /sensors/{id}/measurements/latest` to get current reading.

> ***NOTE***
>
> I could hide these sequential requests to find rooms, sensors, and measurements behind a single HTTP endpoint and call it a "REST API" but that does not strictly align to the definition of REST.

Using an ODATA API, the client can use the powerful `$expand` feature to traverse relationships in a single request:

```
GET /rooms?$filter=floor eq '2'&$expand=sensors($expand=latestMeasurement)
```

This improves the efficiency by reducing requests to one, but the server determines the ultimate depth and shape of the response based on the expansion logic.

Using a GrapQL API, the client sends one precise query that dictates the exact shape and fields of the required response.